Problem 01 :

// left shift

Sample Test Cases

Sl	Input Array	Process	Output Array
01	[5, 10, 15, 20]	১ ঘর বামে সরবে	[10, 15, 20, 5]
02	[100, 200, 300]	১ ঘর বামে সরবে	[200, 300, 100]

Java Solution (সরাসরি মেইন মেথডে**)**

Java

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        // স্ক্যানার অবজেক্ট তৈরি ইউজার থেকে ইনপুট নেওয়ার জন্য
        Scanner sc = new Scanner(System.in);

        // অ্যারের সাইজ ইনপুট নেওয়া
        System.out.print("Enter array size: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        // অ্যারের উপাদানগুলো ইনপুট নেওয়া
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // --- শিফটিং লজিক শুরু ---

        if (n > 0) {
            // ১. প্রথম উপাদানটি সেভ করে রাখা যাতে হারিয়ে না যায়
            int temp = arr[0];
```

```
            // ২. লুপ চালিয়ে প্রতিটি এলিমেন্টকে এক ঘর বামে সরানো
            for (int i = 0; i < n - 1; i++) {
                arr[i] = arr[i + 1]; // i+1 নম্বর ঘরের মান i নম্বর ঘরে আসবে
            }

            // ৩. সেভ করে রাখা প্রথম মানটি একদম শেষে বসিয়ে দেওয়া
            arr[n - 1] = temp;
        }

        // --- শিফটিং লজিক শেষ ---

        // রেজাল্ট প্রিন্ট করা
        System.out.println("After Left Shift:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

কেন এই কোডটি কাজ করছে?

- সংরক্ষণ: শুরুতে `temp = arr[0]` করার মাধ্যমে আমরা প্রথম মানটিকে সুরক্ষিত করি।
- স্থানান্তর: লুপের ভেতরে `arr[i] = arr[i + 1]` লাইনটি দ্বিতীয় মানকে প্রথম ঘরে, তৃতীয় মানকে দ্বিতীয় ঘরে—এভাবে সিরিয়ালি এক ঘর বামে নিয়ে আসে।
- শেষ ধাপ: লুপ শেষ হওয়ার পর অ্যারের শেষ ইনডেক্সটি খালি থাকে (বা ডুপ্লিকেট মান থাকে), সেখানে আমরা শুরুতে রাখা `temp` এর মানটি বসিয়ে দিই।

Problem 02 👍
// right shift

অ্যারে রাইট শিফট (Right Shift) করার ক্ষেত্রে লজিকটি লেফট শিফটের ঠিক উল্টো। এখানে প্রতিটি উপাদান এক ঘর ডানে সরে যাবে এবং শেষের উপাদানটি শুরুতে চলে আসবে।

নিচে ২ টি স্যাম্পল ইনপুট-আউটপুট এবং জাভা কোড দেওয়া হলো:

Sample Test Cases (Right Shift)

Case	Input Array	Process	Output Array
01	[10, 20, 30, 40]	১ ঘর ডানে সরবে	[40, 10, 20, 30]
02	[1, 2, 3, 4, 5]	১ ঘর ডানে সরবে	[5, 1, 2, 3, 4]

Java Solution (Right Shift)

রাইট শিফটের সময় লুপটি অ্যারের শেষ দিক থেকে শুরু করতে হয়, যাতে মানগুলো হারিয়ে না যায়।

Java

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // অ্যারের সাইজ ইনপুট নেওয়া
        System.out.print("Enter array size: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        // অ্যারের উপাদানগুলো ইনপুট নেওয়া
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // --- রাইট শিফটিং লজিক শুরু ---

        if (n > 0) {
            // ১. শেষ উপাদানটি একটি ভ্যারিয়েবলে আলাদা করে রাখা
            int last = arr[n - 1];

            // ২. লুপটি উল্টো দিক (শেষ থেকে) চালিয়ে এক ঘর ডানে সরানো
            // এখানে i শুরু হবে শেষ ইনডেক্সের আগেরটা থেকে (n-2)
```

```
            for (int i = n - 2; i >= 0; i--) {
                arr[i + 1] = arr[i]; // i নম্বর ঘরের মান i+1 ঘরে নিয়ে যাওয়া
            }

            // ৩. আলাদা করে রাখা শেষ মানটি একদম শুরুতে বসিয়ে দেওয়া
            arr[0] = last;
        }

        // --- রাইট শিফটিং লজিক শেষ ---

        // রেজাল্ট প্রিন্ট করা
        System.out.println("After Right Shift:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

কেন এখানে উল্টো লুপ **(Backward Loop)** ব্যবহার করা হয়েছে?

রাইট শিফটের সময় আমরা যদি শুরু থেকে লুপ চালাই (`arr[1] = arr[0]`), তবে দ্বিতীয় ঘরের আসল মানটি হারিয়ে যাবে। তাই আমরা শেষ থেকে কাজ শুরু করি:

1. শেষের মানটি `last` ভ্যারিয়েবলে রাখি।
2. শেষের ঘরে তার আগের ঘরের মান বসাই।
3. এভাবে একদম প্রথম ঘর পর্যন্ত পিছিয়ে আসি।
4. সবশেষে প্রথম ঘরে `last` মানটি বসিয়ে দেই।

==========

Problem 03 :

নিচে ২ টি স্যাম্পল এবং কোড দেওয়া হলো:

Sample Test Cases (Multi-position Left Shift)

Case	Input Array	Shift Count (k)	Output Array

01	[1, 2, 3, 4, 5]	2	[3, 4, 5, 1, 2]
02	[10, 20, 30, 40]	3	[40, 10, 20, 30]

Java Solution (Rotation Logic)

যেকোনো পজিশনের জন্য শিফট করতে হলে, ১ বার শিফট করার লজিকটিকে একটি বাড়তি লুপের ভেতরে রাখলেই কাজ হয়ে যাবে।

Java

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] arr = {1, 2, 3, 4, 5};
        int n = arr.length;

        System.out.print("How many positions to shift left? ");
        int k = sc.nextInt();

        // যদি k এর মান অ্যারের সাইজের চেয়ে বড় হয়, তবে রোটেশন সহজ করার জন্য % ব্যবহার করা হয়
        k = k % n;

        // ১. বাইরের লুপটি k বার চলবে (যত ঘর সরাতে চাই)
        for (int j = 0; j < k; j++) {

            // ২. ১ বার বামে সরানোর লজিক (আগের মতোই)
            int temp = arr[0];
            for (int i = 0; i < n - 1; i++) {
                arr[i] = arr[i + 1];
            }
            arr[n - 1] = temp;
        }
```

```
        // আউটপুট প্রিন্ট করা
        System.out.println("After " + k + " shifts:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

এই কোডের গুরুত্বপূর্ণ দিক:

- **`k = k % n;`**: আপনার অ্যারে যদি ৫ সাইজের হয় আর আপনি যদি ৫ বার শিফট করেন, তবে অ্যারে আবার আগের মতোই হয়ে যাবে। তাই অপ্রয়োজনীয় লুপ চালানো বন্ধ করতে আমরা রিমেইন্ডার বা ভাগশেষ ব্যবহার করি।
- **Nested Loop**: এখানে বাইরের লুপটি ঠিক করে দেয় কতবার শিফটিং হবে, আর ভেতরের লুপটি প্রতিটি উপাদানকে এক ঘর করে সরিয়ে দেয়।

==========

Problem 04 👍

চমৎকার! আপনি যদি যেকোনো নির্দিষ্ট পজিশন বা **k** সংখ্যক বার ডান দিকে **(Right Shift)** সরাতে চান, তবে লজিকটি হবে নিচের মতো:

Sample Test Cases (Multi-position Right Shift)

Case	Input Array	Shift Count (k)	Output Array
01	[1, 2, 3, 4, 5]	2	[4, 5, 1, 2, 3]
02	[10, 20, 30]	1	[30, 10, 20]

Java Solution (Directly in Main Method)

এখানে আমরা ইউজার থেকে নেওয়া ইনপুট অনুযায়ী লুপ ঘুরিয়ে ডান দিকে শিফট করবো।

Java

```
import java.util.Scanner;

public class Main {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // প্রাথমিক অ্যারে
        int[] arr = {1, 2, 3, 4, 5};
        int n = arr.length;

        System.out.print("কয় ঘর ডান দিকে সরাতে চান? ");
        int k = sc.nextInt();

        // অপ্রয়োজনীয় লুপ এড়াতে রিমেইন্ডার বের করা (যেমন ৫ সাইজের অ্যারে ৫ বার ঘুরালে আগের মতোই থাকে)
        k = k % n;

        // ১. বাইরের লুপটি k বার চলবে
        for (int j = 0; j < k; j++) {

            // ২. ১ বার ডান দিকে সরানোর লজিক
            int last = arr[n - 1]; // শেষ এলিমেন্টটি সেভ রাখা

            // শেষ থেকে লুপ চালিয়ে মানগুলো ডানে সরানো
            for (int i = n - 2; i >= 0; i--) {
                arr[i + 1] = arr[i];
            }

            arr[0] = last; // সেভ করা মানটি শুরুতে বসানো
        }

        // আউটপুট প্রিন্ট
        System.out.println(k + " বার রাইট শিফট করার পর অ্যারে:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

কোডটি যেভাবে কাজ করছে:

- **k % n** কেন?: মনে করুন অ্যারের সাইজ ৫, আর আপনি ১০ বার শিফট করতে বললেন। ১০ বার শিফট করা আর ০ বার শিফট করা একই কথা। তাই % অপারেটর কোডকে ফাস্ট করে।
- **Nested Loop (**লুপের ভেতর লুপ**)**: বাইরের লুপটি নির্ধারণ করে আপনি মোট কতবার এই শিফটিং প্রক্রিয়াটি চালাবেন।

- **Backwards Loop**: রাইট শিফটের সময় সবসময় `i = n - 2` থেকে শুরু করে পিছনের দিকে লুপ চালাতে হয়, নাহলে মানগুলো ডুপ্লিকেট হয়ে যায়।

==========

সহজ কথায় বলতে গেলে, সাধারণ প্রোগ্রামিং সমস্যার ক্ষেত্রে **Rotate** এবং **Shift** প্রায় একই রকম মনে হলেও এদের মধ্যে একটি সূক্ষ্ম পার্থক্য আছে।

নিচে পার্থক্যটি পরিষ্কার করে দেওয়া হলো:

১. **Shift** (শিফট)

শিফটিং-এর সময় যখন কোনো উপাদান একপাশ থেকে বের হয়ে যায়, তখন সেই জায়গাটি খালি হয়ে যায় (সাধারণত সেখানে `0` বা অন্য কোনো ডিফল্ট মান বসে)। এটি সার্কুলার নয়।

- **Left Shift:** `[1, 2, 3]` কে ১ ঘর শিফট করলে হবে `[2, 3, 0]`। (১ হারিয়ে গেছে)
- **Right Shift:** `[1, 2, 3]` কে ১ ঘর শিফট করলে হবে `[0, 1, 2]`। (৩ হারিয়ে গেছে)

২. **Rotate** (রোটেট)

রোটেশন মানে হলো উপাদানটি একপাশ থেকে বের হয়ে অন্যপাশে আবার ফিরে আসবে। এটি একটি চক্র বা সার্কেলের মতো কাজ করে। আপনার করা আগের প্রবলেমগুলো আসলে রোটেশন ছিল।

- **Left Rotate:** `[1, 2, 3]` কে ১ ঘর রোটেট করলে হবে `[2, 3, 1]`। (১ শেষে চলে এসেছে)
- **Right Rotate:** `[1, 2, 3]` কে ১ ঘর রোটেট করলে হবে `[3, 1, 2]`। (৩ শুরুতে চলে এসেছে)

মূল পার্থক্য একনজরে:

বৈশিষ্ট্য	**Shift** (শিফট)	**Rotate** (রোটেট)
ফলাফল	ডেটা হারিয়ে যেতে পারে	কোনো ডেটা হারায় না

খালি জায়গা	শূন্য (0) বা নাল (null) দিয়ে পূর্ণ হয়	অন্য প্রান্তের মান দিয়ে পূর্ণ হয়
ধরন	এটি লিনিয়ার (Linear)	এটি সাইক্লিক (Cyclic)

নোট: ইন্টারভিউ বা পরীক্ষার প্রশ্নে অনেক সময় "Shift Left" বললেও তারা আসলে রোটেশনই বোঝাতে চায় (যেমন: প্রথম এলিমেন্ট শেষে নেওয়া)। তাই প্রশ্নটি ভালো করে পড়ে দেখা উচিত যে মানটি কি হারিয়ে যাচ্ছে (Shift) নাকি অন্য প্রান্তে যাচ্ছে (Rotate)।

==========

Problem 05 :

, আমরা অরিজিনাল অ্যারেকে পরিবর্তন না করে একটি নতুন অ্যারে তৈরি করে সেখানে শুধু ইউনিক (Unique) মানগুলো রাখবো। এটি অনেক বেশি ক্লিন পদ্ধতি।

নিচে ২ টি স্যাম্পল এবং কোড দেওয়া হলো:

Sample Test Cases

Case	Input Array (Original)	Unique Elements Found	Output (New Array)
01	[5, 1, 5, 2, 1]	1, 2, 5	[1, 2, 5]
02	[7, 7, 7]	7	[7]

Java Solution (Using a New Array)

এই পদ্ধতিতে প্রথমে আমরা সর্ট করবো, তারপর কয়টি ইউনিক নম্বর আছে তা গুনে নিয়ে ঠিক সেই সাইজের একটি নতুন অ্যারে তৈরি করবো।

```
Java
public class Main {
    public static void main(String[] args) {
        int[] oldArr = {12, 5, 12, 7, 5, 10};
        int n = oldArr.length;

        // --- ধাপ ১: Bubble Sort (পুরানো অ্যারেকে সাজানো) ---
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
```

```
                if (oldArr[j] > oldArr[j + 1]) {
                    int temp = oldArr[j];
                    oldArr[j] = oldArr[j + 1];
                    oldArr[j + 1] = temp;
                }
            }
        }
        // সর্ট করার পর: {5, 5, 7, 10, 12, 12}

        // --- ধাপ ২: ইউনিক এলিমেন্ট কয়টি আছে তা গণনা করা ---
        int uniqueCount = 0;
        for (int i = 0; i < n; i++) {
            // যদি এটি শেষ এলিমেন্ট হয় অথবা বর্তমান এলিমেন্ট পরেরটির সমান না হয়
            if (i < n - 1 && oldArr[i] == oldArr[i + 1]) {
                continue; // ডুপ্লিকেট হলে স্কিপ করি
            }
            uniqueCount++;
        }

        // --- ধাপ ৩: নতুন অ্যারে তৈরি এবং মান বসানো ---
        int[] newArr = new int[uniqueCount];
        int index = 0;

        for (int i = 0; i < n; i++) {
            if (i < n - 1 && oldArr[i] == oldArr[i + 1]) {
                continue;
            }
            newArr[index] = oldArr[i]; // ইউনিক মানটি নতুন অ্যারেতে রাখা
            index++;
        }

        // --- ফলাফল প্রিন্ট ---
        System.out.println("New Array (Unique Elements Only):");
        for (int i = 0; i < newArr.length; i++) {
            System.out.print(newArr[i] + " ");
        }
    }
}
```

এই সমাধানের সুবিধা কী?

1. **Memory Management:** আমরা `newArr` তৈরি করেছি ঠিক ততটুকু সাইজ দিয়ে যতটুকু প্রয়োজন (`uniqueCount`)। এতে মেমরি অপচয় হয় না।
2. **Originality:** আপনার কাছে এখন দুটি অ্যারে আছে—একটি অরিজিনাল (সর্ট করা অবস্থায়) এবং একটি শুধুমাত্র ইউনিক মান নিয়ে তৈরি নতুন অ্যারে।
3. **Accuracy:** সর্ট করার ফলে ডুপ্লিকেটগুলো পাশাপাশি থাকে, তাই `if (oldArr[i] == oldArr[i + 1])` লজিকটি দিয়ে খুব সহজে ডুপ্লিকেট এড়িয়ে যাওয়া যায়।

==========

Problem 06 👍

// move zero to the end

অ্যারের সব জিরো (0) শেষে নিয়ে যাওয়ার সমস্যাটি সমাধান করার সবচেয়ে সহজ উপায় হলো **Two-Pointer** লজিক ব্যবহার করা। এখানে আমরা কোনো বিল্ট-ইন মেথড বা নতুন অ্যারে ব্যবহার না করেই সরাসরি মেইন অ্যারেতে কাজ করবো।
নিচে আপনার জন্য সমাধানটি দেওয়া হলো:

Sample Test Cases

Sl	Input Array	Non-zero Order	Output Array
01	[0, 1, 0, 3, 12]	1, 3, 12	[1, 3, 12, 0, 0]
02	[4, 0, 5, 0, 0, 2]	4, 5, 2	[4, 5, 2, 0, 0, 0]

Java Solution (Moving Zeros to End)

Java
```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 0, 4, 3, 0, 5, 0};
        int n = arr.length;

        // ১. একটি ভ্যারিয়েবল রাখা যা ট্র্যাক করবে কোথায় নন-জিরো সংখ্যা বসবে
        int count = 0;

        // ২. অ্যারেটি একবার লুপ চালিয়ে চেক করা
        for (int i = 0; i < n; i++) {
            // যদি সংখ্যাটি জিরো না হয়
            if (arr[i] != 0) {
                // বর্তমান নন-জিরো সংখ্যাটিকে 'count' পজিশনে নিয়ে আসা
                arr[count] = arr[i];
                count++; // পরবর্তী জায়গার জন্য count বাড়ানো
            }
        }

        // ৩. লুপ শেষ হওয়ার পর count থেকে n পর্যন্ত সব ঘরে জিরো বসিয়ে দেওয়া
```

```
        while (count < n) {
            arr[count] = 0;
            count++;
        }

        // আউটপুট প্রিন্ট করা
        System.out.println("After moving zeros to end:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

কাজ করার প্রক্রিয়া **(Step-by-Step):**

1. প্রথম লুপ**:** আমরা যখনই কোনো নন-জিরো সংখ্যা (যেমন ১, ২, ৩...) পাই, তখনই তাকে অ্যারের শুরুর দিকে পাঠিয়ে দিই। `count` ভ্যারিয়েবলটি মনে রাখে পরবর্তী নন-জিরো সংখ্যাটি কোথায় বসবে।
2. অর্ডার ঠিক রাখা**:** যেহেতু আমরা অ্যারের শুরু থেকে লুপ চালাচ্ছি, তাই নন-জিরো সংখ্যাগুলোর সিরিয়াল বা রিলেটিভ অর্ডার (যেমন ১ এর পর ২, তারপর ৪) ঠিক থাকে।
3. বাকি ঘর পূরণ**:** প্রথম লুপ শেষ হওয়ার পর `count` এর মান নির্দেশ করে কয়টি নন-জিরো সংখ্যা পাওয়া গেছে। এরপর অ্যারের বাকি যতগুলো ঘর খালি আছে, সেগুলোতে সিম্পলি `0` বসিয়ে দিলেই সব জিরো শেষে চলে যায়।

app 02 :
Problem 07

Swap (জায়গা বদল**)** পদ্ধতি ব্যবহার করলে আমাদের দ্বিতীয়বার কোনো লুপ চালিয়ে জিরো বসাতে হয় না। এক লুপেই সব কাজ হয়ে যায়।

নিচে Swap পদ্ধতি ব্যবহার করে কোডটি দেওয়া হলো:

Java Solution (Using Swap - One Pass)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 0, 4, 3, 0, 5, 0};
        int n = arr.length;

        // count ভ্যারিয়েবলটি ট্র্যাক করবে সর্বশেষ জিরো কোথায় আছে
        int count = 0;

        for (int i = 0; i < n; i++) {
```

```
            // যদি সংখ্যাটি জিরো না হয়, তবে একে count ইনডেক্সের সাথে অদলবদল (swap) করি
            if (arr[i] != 0) {
                // Swap logic
                int temp = arr[i];
                arr[i] = arr[count];
                arr[count] = temp;

                // swap করার পর count এক ঘর ডানে সরিয়ে দেই
                count++;
            }
        }

        // আউটপুট প্রিন্ট
        System.out.println("One-pass Swap ব্যবহার করে রেজাল্ট:");
        for (int val : arr) {
            System.out.print(val + " ");
        }
    }
}
```

এই পদ্ধতির বিশেষত্ব কী?

১. একবার লুপ **(One Pass):** আগের কোডে আমরা প্রথমে সংখ্যাগুলো বসিয়েছি, তারপর খালি ঘরে জিরো বসিয়েছি। কিন্তু এখানে এক লুপেই সব জিরো পেছনে চলে যাচ্ছে। ২. মেমোরি সেভ**:** কোনো বাড়তি অ্যারে লাগছে না এবং কাজ দ্রুত হচ্ছে। ৩. লজিক**:** যখনই লুপ কোনো নন-জিরো সংখ্যা পায়, সেটিকে `count` ইনডেক্সে থাকা জিরোর সাথে অদলবদল করে ফেলে। এতে জিরোগুলো ধীরে ধীরে ডান দিকে সরে যায়।

==========
Problem 08 :
Move zero to the front
সব জিরো শুরুতে **(Front)** নিয়ে আসা ঠিক আগের সমস্যার উল্টো। এখানে আমরা অ্যারের শেষ থেকে লুপ চালানো শুরু করবো যাতে নন-জিরো সংখ্যাগুলো তাদের সঠিক ক্রম বজায় রেখে ডানে সরে যায় এবং জিরোগুলো বামে চলে আসে।

নিচে সমাধানটি দেওয়া হলো:

Sample Test Case

- **Input:** `[1, 2, 0, 4, 3, 0, 5, 0]`
- **Output:** `[0, 0, 0, 1, 2, 4, 3, 5]`

Java Solution (Move Zeros to Front)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 0, 4, 3, 0, 5, 0};
        int n = arr.length;

        // ১. count ভ্যারিয়েবলটি অ্যারের একদম শেষ ইনডেক্সে সেট করি
        int count = n - 1;

        // ২. অ্যারের শেষ থেকে শুরু পর্যন্ত লুপ চালাই
        for (int i = n - 1; i >= 0; i--) {
            // যদি সংখ্যাটি জিরো না হয়
            if (arr[i] != 0) {
                // একে শেষের দিক থেকে বসানো শুরু করি
                int temp = arr[i];
                arr[i] = arr[count];
                arr[count] = temp;

                // বসানোর পর count এক ঘর বামে সরিয়ে আনি
                count--;
            }
        }

        // আউটপুট প্রিন্ট করা
        System.out.println("সব জিরো শুরুতে নিয়ে আসার পর অ্যারে:");
        for (int val : arr) {
            System.out.print(val + " ");
        }
    }
}
```

এই কোডের লজিক:

- **Reverse Loop (**পিছন থেকে লুপ**):** যেহেতু আমাদের জিরো শুরুতে আনতে হবে, তাই নন-জিরো সংখ্যাগুলোকে অ্যারের শেষে পাঠিয়ে দিতে হবে। এ জন্য আমরা পিছন থেকে কাজ শুরু করেছি।
- **Swap (**অদলবদল**):** যখনই আমরা কোনো নন-জিরো সংখ্যা পাই, সেটিকে `count` পজিশনে থাকা জিরোর সাথে অদলবদল করে দিই। এতে সব জিরো ধীরে ধীরে অ্যারের সামনের দিকে (বামে) জমা হয়।

- **Relative Order:** যেহেতু আমরা পিছন থেকে সিরিয়াল মেইনটেইন করছি, তাই `1, 2, 4, 3, 5` এর ক্রম ঠিক থাকে।

==========

Problem 09

2nd largest

অ্যারের সেকেন্ড লার্জেস্ট (Second Largest) এলিমেন্ট বের করার সবচেয়ে দক্ষ পদ্ধতি হলো একটি লুপের মাধ্যমে একসাথেই **Largest** এবং **Second Largest** ট্র্যাক করা। এতে অ্যারে সর্ট করার প্রয়োজন হয় না, ফলে কোড অনেক দ্রুত কাজ করে।
নিচে আপনার জন্য সমাধানটি দেওয়া হলো:

Sample Test Cases

Input Array	Largest	Second Largest	Output
[12, 35, 1, 10, 34, 1]	35	34	34
[10, 5, 10]	10	5	5
[10, 10, 10]	10	নেই	-1

Java Solution (One-Pass Logic)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {12, 35, 1, 10, 34, 1};
        int n = arr.length;

        // ১. শুরুতে দুটি ভ্যারিয়েবলকে -১ দিয়ে ডিক্লেয়ার করা (যেহেতু সব পজিটিভ সংখ্যা)
        int largest = -1;
        int secondLargest = -1;

        // ২. পুরো অ্যারে একবার লুপ চালিয়ে চেক করা
        for (int i = 0; i < n; i++) {
```

```
            // যদি বর্তমান সংখ্যাটি সবচাইতে বড় সংখ্যার চেয়েও বড় হয়
            if (arr[i] > largest) {
                // আগের সবচাইতে বড় সংখ্যাটি এখন সেকেন্ড লার্জেস্ট হয়ে যাবে
                secondLargest = largest;
                // বর্তমান সংখ্যাটি হবে সবচাইতে বড়
                largest = arr[i];
            }
            // যদি সংখ্যাটি largest এর সমান না হয় কিন্তু secondLargest এর চেয়ে বড় হয়
            else if (arr[i] > secondLargest && arr[i] != largest) {
                secondLargest = arr[i];
            }
        }

        // ফলাফল প্রিন্ট করা
        System.out.println("Second Largest Element: " + secondLargest);
    }
}
```

এই লজিকটি যেভাবে কাজ করে:

- **Largest Update:** যদি আমরা বর্তমান `largest` এর চেয়ে বড় কোনো সংখ্যা পাই, তার মানে আমাদের কাছে এখন নতুন রাজা (Largest) আছে। তবে পুরনো রাজা কিন্তু একদম বাদ যাবে না, সে এখন `secondLargest` এর পদে বসবে।
- **Second Largest Update:** যদি কোনো সংখ্যা `largest` এর সমান না হয় (কারণ আমাদের distinct বা আলাদা সংখ্যা দরকার), কিন্তু বর্তমান `secondLargest` এর চেয়ে বড় হয়, তবে আমরা শুধুমাত্র `secondLargest` আপডেট করি।
- **Distinct Check:** `arr[i] != largest` কন্ডিশনটি নিশ্চিত করে যে আমরা ডুপ্লিকেট বড় সংখ্যা (যেমন `[10, 10, 5]` এর ক্ষেত্রে ১০ কে দুইবার বড় ধরবে না) হিসেব করছি না।

==========

Problem 10 :

অ্যারেটি সর্ট **(Sorting)** করে সমাধান করতে চান, তবে লজিকটি হবে এরকম: প্রথমে অ্যারেটিকে বড় থেকে ছোট (Descending Order) সাজাতে হবে, তারপর প্রথম এলিমেন্টটি থেকে আলাদা যে প্রথম ছোট সংখ্যাটি পাওয়া যাবে, সেটিই হবে সেকেন্ড লার্জেস্ট।

নিচে সর্টিং ব্যবহার করে সমাধান দেওয়া হলো:

Java Solution (Using Sorting)

Java

```
public class Main {
```

```
    public static void main(String[] args) {
        int[] arr = {12, 35, 1, 10, 34, 1};
        int n = arr.length;

        // ১. বাবল সর্ট ব্যবহার করে বড় থেকে ছোট (Descending) সাজানো
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] < arr[j + 1]) { // '<' চিহ্ন দিলে বড় থেকে ছোট হবে
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        // সর্ট করার পর অ্যারে: {35, 34, 12, 10, 1, 1}

        // ২. সেকেন্ড লার্জেস্ট খোঁজা (প্রথম এলিমেন্ট থেকে আলাদা হতে হবে)
        int largest = arr[0];
        int secondLargest = -1;

        for (int i = 1; i < n; i++) {
            if (arr[i] != largest) {
                secondLargest = arr[i];
                break; // প্রথম যে আলাদা ছোট সংখ্যা পাবো সেটিই সেকেন্ড লার্জেস্ট
            }
        }

        // ফলাফল প্রিন্ট
        System.out.println("Second Largest Element is: " + secondLargest);
    }
}
```

এই পদ্ধতির সুবিধা ও অসুবিধা:

- সুবিধা: এটি বোঝা খুব সহজ। বড় থেকে ছোট সাজালে প্রথম কয়েকটা এলিমেন্ট চেক করলেই উত্তর পাওয়া যায়।
- অসুবিধা: এটি অনেক স্লো (Time Complexity বেশি)। আপনার অ্যারে যদি অনেক বড় হয় (যেমন ১ লক্ষ সংখ্যা), তবে সর্ট করতে অনেক সময় লাগবে। সে ক্ষেত্রে আগের দেওয়া **One-Pass Logic** টাই সবথেকে সেরা।

==========

Problem 11 :
থার্ড লার্জেস্ট (Third Largest) এলিমেন্ট বের করার লজিকটি সেকেন্ড লার্জেস্টের মতোই, তবে এখানে আমাদের তিনটি ভ্যারিয়েবল ট্র্যাক করতে হবে। যেহেতু প্রশ্নে বলা হয়েছে সব উপাদান **distinct** (আলাদা), তাই ডুপ্লিকেট চেক করার ঝামেলা নেই।

নিচে সহজ সমাধানটি দেওয়া হলো:

Sample Test Cases

Input Array	1st Largest	2nd Largest	3rd Largest	Output
{1, 14, 2, 16, 10, 20}	20	16	14	14
{19, -10, 20, 14, 2}	20	19	14	14

Java Solution (Efficient One-Pass)

এই পদ্ধতিতে আমরা মাত্র একবার লুপ চালিয়ে প্রথম, দ্বিতীয় এবং তৃতীয় বড় সংখ্যাগুলো খুঁজে বের করবো।

Java
```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 14, 2, 16, 10, 20};
        int n = arr.length;

        // যদি অ্যারেতে ৩টির কম উপাদান থাকে
        if (n < 3) {
            System.out.println("Invalid Input (Size should be at least 3)");
            return;
        }

        // ১. তিনটি ভ্যারিয়েবলকে অনেক ছোট মান দিয়ে শুরু করা
        int first = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;
        int third = Integer.MIN_VALUE;
```

```
        // ২. পুরো অ্যারে একবার চেক করা
        for (int i = 0; i < n; i++) {

            // যদি বর্তমান সংখ্যাটি ফার্স্ট এর চেয়ে বড় হয়
            if (arr[i] > first) {
                third = second;     // দ্বিতীয়টি হয়ে যাবে তৃতীয়
                second = first;     // প্রথমটি হয়ে যাবে দ্বিতীয়
                first = arr[i];     // বর্তমানটি হবে প্রথম
            }
            // যদি ফার্স্ট এর চেয়ে ছোট কিন্তু সেকেন্ড এর চেয়ে বড় হয়
            else if (arr[i] > second) {
                third = second;
                second = arr[i];
            }
            // যদি সেকেন্ড এর চেয়ে ছোট কিন্তু থার্ড এর চেয়ে বড় হয়
            else if (arr[i] > third) {
                third = arr[i];
            }
        }

        // ফলাফল প্রিন্ট
        System.out.println("The third largest element is: " + third);
    }
}
```

এই লজিকটি যেভাবে কাজ করছে **(Step-by-Step):**

- লেভেল ১ **(First):** যখনই বড় কোনো সংখ্যা পাওয়া যায়, সেটি আগের সব রেকর্ড ভেঙে ফেলে। তাই থার্ড পায় সেকেন্ডের মান, সেকেন্ড পায় ফার্স্টের মান এবং ফার্স্ট নিজে আপডেট হয়।
- লেভেল ২ **(Second):** যদি সংখ্যাটি ফার্স্টের চেয়ে বড় না হয় কিন্তু সেকেন্ডের চেয়ে বড় হয়, তবে শুধুমাত্র থার্ড এবং সেকেন্ড আপডেট হবে।
- লেভেল ৩ **(Third):** যদি সংখ্যাটি শুধু থার্ডের চেয়ে বড় হয়, তবে শুধু থার্ড আপডেট হবে।
- **Integer.MIN_VALUE:** এটি জাভার সবচেয়ে ছোট ইন্টিজার মান। আমরা এটি ব্যবহার করেছি যাতে অ্যারেতে নেগেটিভ সংখ্যা থাকলেও কোডটি সঠিকভাবে কাজ করে।

==========

Problem 12

অ্যারে রিভার্স (Reverse) করার সবচাইতে কার্যকর পদ্ধতি হলো **Two-Pointer Approach**। এই পদ্ধতিতে আমরা অ্যারের শুরু এবং শেষ থেকে দুটি পয়েন্টার ধরে একে অপরের সাথে মানগুলো অদলবদল (Swap) করি।

নিচে আপনার জন্য সমাধানটি দেওয়া হলো:

Sample Test Cases

Case	Input Array	Process	Output Array
01	[1, 4, 3, 2, 6, 5]	১↔৫, ৪↔৬, ৩↔২	[5, 6, 2, 3, 4, 1]
02	[4, 5, 1, 2]	৪↔২, ৫↔১	[2, 1, 5, 4]

Java Solution (Two-Pointer Approach)

এই পদ্ধতিতে কোনো বাড়তি অ্যারে প্রয়োজন হয় না, তাই এটি মেমোরি সাশ্রয়ী।

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 4, 3, 2, 6, 5};
        int n = arr.length;

        // ১. শুরু (start) এবং শেষ (end) পয়েন্টার সেট করা
        int start = 0;
        int end = n - 1;

        // ২. যতক্ষণ না মাঝখানে পৌঁছাচ্ছি, ততক্ষণ swap চলবে
        while (start < end) {
            // মান অদলবদল (Swap) করার লজিক
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;

            // ৩. পয়েন্টার দুটিকে এগিয়ে এবং পিছিয়ে নেওয়া
            start++; // শুরু থেকে সামনে যাবে
            end--;   // শেষ থেকে পিছনে আসবে
        }

        // আউটপুট প্রিন্ট করা
        System.out.println("Reversed Array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
```

```
        }
    }
}
```

কোডটি যেভাবে কাজ করছে **(Logic):**

- **Swap Logic:** আমরা `arr[0]` এর সাথে `arr[n-1]` অদলবদল করি, তারপর `arr[1]` এর সাথে `arr[n-2]` এবং এভাবেই চলতে থাকে।
- **While Loop Condition:** `start < end` থাকা পর্যন্তই লুপ চলবে। যখন `start` এবং `end` একে অপরকে অতিক্রম করে যাবে বা সমান হবে, তখন বোঝা যাবে পুরো অ্যারে উল্টানো হয়ে গেছে।
- **No Extra Space:** এখানে আমরা কোনো নতুন অ্যারে তৈরি করিনি, বরং একই অ্যারের ভেতরে মানগুলো বদলে দিয়েছি। একে **In-place algorithm** বলা হয়।

==========

Problem 13

অ্যারের উপাদানগুলোকে একটি নির্দিষ্ট সাইজ k অনুযায়ী গ্রুপ করে রিভার্স করার জন্য আমাদের মূল রিভার্স লজিকটিকেই একটি লুপের ভেতর ব্যবহার করতে হবে যা প্রতিবার k ঘর পর পর লাফ দিয়ে (Jump) কাজ করবে।

নিচে আপনার জন্য সমাধানটি দেওয়া হলো:

Sample Test Cases

Input Array	k	Groups to Reverse	Output Array
[1, 2, 3, 4, 5, 6, 7, 8]	3	(1,2,3), (4,5,6), (7,8)	[3, 2, 1, 6, 5, 4, 8, 7]
[1, 2, 3, 4, 5]	3	(1,2,3), (4,5)	[3, 2, 1, 5, 4]

Java Solution (In-place Group Reversal)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6, 7, 8};
        int k = 3;
        int n = arr.length;
```

```
        // ১. মেইন লুপটি k ঘর পর পর লাফ দিবে
        for (int i = 0; i < n; i += k) {

            int left = i; // প্রতি গ্রুপের শুরুর ইনডেক্স

            // ২. গ্রুপের শেষ ইনডেক্স খুঁজে বের করা
            // যদি (i + k - 1) অ্যারের সাইজ ছাড়িয়ে যায়, তবে শেষ ইনডেক্স হবে (n - 1)
            int right = Math.min(i + k - 1, n - 1);

            // ৩. গ্রুপটিকে রিভার্স করার জন্য Two-pointer swap
            while (left < right) {
                int temp = arr[left];
                arr[left] = arr[right];
                arr[right] = temp;

                left++;
                right--;
            }
        }

        // ফলাফল প্রিন্ট
        System.out.println("Group Reverse করার পর অ্যারে:");
        for (int val : arr) {
            System.out.print(val + " ");
        }
    }
}
```

লজিকটি যেভাবে কাজ করছে:

- **`i += k`**: এই লুপটি সাধারণ লুপের মতো ১ ঘর করে না এগিয়ে k ঘর করে সামনে আগায়। যেমন $k=3$ হলে এটি ইনডেক্স 0, তারপর 3, তারপর 6 এ যাবে।
- **`Math.min(i + k - 1, n - 1)`**: এটি খুব গুরুত্বপূর্ণ অংশ। যখন আমাদের গ্রুপের সাইজ k এর চেয়ে কম থাকে (যেমন উদাহরণের শেষ গ্রুপ ৮, ৭), তখন যাতে **`ArrayIndexOutOfBoundsException`** না হয়, সে জন্য এটি অ্যারের শেষ সীমানা (`n-1`) নিশ্চিত করে।
- **In-place Swap**: লুপের ভেতরে আমরা সেই নির্দিষ্ট গ্রুপের `left` এবং `right` পয়েন্টার ব্যবহার করে মানগুলো উল্টে দিচ্ছি।

==========

Problem 14

অ্যারের সব সাব-অ্যারের যোগফল (Sum of all Subarrays) বের করার দুটি পদ্ধতি আছে। একটি হলো সাধারণ ৩টি লুপ ব্যবহার করে (Brute Force), আর অন্যটি হলো একটি ফর্মুলা ব্যবহার করে খুব দ্রুত সমাধান করা।

নিচে সহজ এবং দক্ষ (Efficient) সমাধানটি দেওয়া হলো:

Sample Test Case

Input: `arr[] = [1, 2, 3]` **Subarrays:**

- `[1]`, `[1, 2]`, `[1, 2, 3]` (যোগফল: 1 + 3 + 6 = 10)
- `[2]`, `[2, 3]` (যোগফল: 2 + 5 = 7)
- `[3]` (যোগফল: 3) **Total Sum:** 10 + 7 + 3 = **20**

Java Solution (Efficient Logic)

এই পদ্ধতিতে আমরা হিসেব করি প্রতিটি সংখ্যা কতবার কতগুলো সাব-অ্যারেতে আসতে পারে। একটি সংখ্যা `arr[i]` মোট `(i + 1) * (n - i)` সংখ্যক সাব-অ্যারেতে থাকে।

```
Java
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4};
        int n = arr.length;
        long totalSum = 0;

        // ১. একটি লুপ চালিয়ে প্রতিটি এলিমেন্টের অবদান বের করা
        for (int i = 0; i < n; i++) {

            // ফর্মুলা: একটি এলিমেন্ট কতবার সাব-অ্যারেতে উপস্থিত থাকে
            // frequency = (শুরুর দিক থেকে অপশন) * (শেষের দিক থেকে অপশন)
            long frequency = (long) (i + 1) * (n - i);

            // এলিমেন্টটির মানকে তার ফ্রিকুয়েন্সি দিয়ে গুণ করে মোট যোগফলে যোগ করা
            totalSum += (arr[i] * frequency);
        }

        // ফলাফল প্রিন্ট করা
        System.out.println("Sum of all possible subarrays: " + totalSum);
    }
}
```

কেন এই ফর্মুলাটি ব্যবহার করলাম?

আমরা যদি ৩টি লুপ ব্যবহার করি (O(n3)), তবে বড় অ্যারের ক্ষেত্রে কোডটি অনেক স্লো হয়ে যাবে। কিন্তু এই গাণিতিক পদ্ধতিটি মাত্র একটি লুপে (O(n)) কাজ শেষ করে।

- **i + 1**: এর মানে হলো arr[i] এর আগে এবং এটিসহ মোট কয়টি ঘর থেকে সাব-অ্যারে শুরু হতে পারে।
- **n - i**: এর মানে হলো arr[i] থেকে শুরু করে অ্যারের শেষ পর্যন্ত কতটি ঘরে সাব-অ্যারে শেষ হতে পারে।
- এই দুটি গুণ করলে পাওয়া যায় ওই নির্দিষ্ট সংখ্যাটি মোট কতগুলো সাব-অ্যারের অংশ হিসেবে আছে।

উদাহরণস্বরূপ **(arr = [1, 2, 3]):**

- ১ নম্বর সংখ্যাটি (index 0) আছে: (0+1)*(3-0)=3 টি সাব-অ্যারেতে।
- ২ নম্বর সংখ্যাটি (index 1) আছে: (1+1)*(3-1)=4 টি সাব-অ্যারেতে।
- ৩ নম্বর সংখ্যাটি (index 2) আছে: (2+1)*(3-2)=3 টি সাব-অ্যারেতে।

মোট যোগফল: (1×3)+(2×4)+(3×3)=3+8+9=20।

আপনি কি এই সমস্যার **Brute Force (Nested Loop)** পদ্ধতিটি দেখতে চান নাকি অন্য কোনো নতুন প্রবলেমে যাবো?

==========

Problem 15

সাব-অ্যারের যোগফল বের করার নেস্টেড লুপ বা **Brute Force** পদ্ধতিটি মূলত তিনটি লুপের মাধ্যমে কাজ করে। এটি বুঝতে সহজ কারণ এটি প্রতিটি সাব-অ্যারে আলাদাভাবে তৈরি করে এবং সেগুলোর যোগফল বের করে।

Java Solution (Nested Loop Approach)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4};
        int n = arr.length;
        long totalSum = 0;

        // ১. সাব-অ্যারের শুরুর ইনডেক্স নির্ধারণ (i)
        for (int i = 0; i < n; i++) {

            // ২. সাব-অ্যারের শেষ ইনডেক্স নির্ধারণ (j)
            for (int j = i; j < n; j++) {
```

```
                // ৩. i থেকে j পর্যন্ত সাব-অ্যারের এলিমেন্টগুলো যোগ করা (k)
                int currentSubarraySum = 0;
                for (int k = i; k <= j; k++) {
                    currentSubarraySum += arr[k];
                }

                // প্রতিটি সাব-অ্যারের যোগফল মূল টোটাল সামে যোগ করা
                totalSum += currentSubarraySum;
            }
        }

        System.out.println("Sum of all subarrays (Nested Loop): " + totalSum);
    }
}
```

কোডটি যেভাবে কাজ করে:

- বাইরের লুপ (**i**): এটি নির্ধারণ করে সাব-অ্যারেটি কোথা থেকে শুরু হবে।
- মাঝখানের লুপ (**j**): এটি নির্ধারণ করে সাব-অ্যারেটি কোথায় গিয়ে শেষ হবে।
- ভেতরের লুপ (**k**): এটি `i` থেকে `j` পর্যন্ত প্রত্যেকটি সংখ্যাকে যোগ করে।

এটিকে কি আরও একটু উন্নত **(Optimize)** করা যায়?

হ্যাঁ! ৩টি লুপের বদলে ২টি লুপ ব্যবহার করেও এটি করা সম্ভব। আমরা ভেতরের `k` লুপটি বাদ দিতে পারি কারণ আমরা যখন `j` কে এক ঘর সামনে নিচ্ছি, তখন আগের যোগফলের সাথে নতুন সংখ্যাটি যোগ করলেই কারেন্ট সাব-অ্যারের সাম পাওয়া যায়।

Java
```
// ২ লুপের অপটিমাইজড ভার্সন:
for (int i = 0; i < n; i++) {
    int currentSum = 0;
    for (int j = i; j < n; j++) {
        currentSum += arr[j]; // সাব-অ্যারে [i...j] এর সাম
        totalSum += currentSum;
    }
}
```

টিপস: ইন্টারভিউতে ৩টি লুপের পদ্ধতি দেখালে তারা সাধারণত ২ লুপের বা ১ লুপের (আগের ফর্মুলা পদ্ধতি) সমাধানটি করতে বলে।

==========

Problem 16

অ্যারের সবচাইতে বড় সাব-অ্যারে সাম (Maximum Subarray Sum) বের করার সবচেয়ে জনপ্রিয় এবং দক্ষ পদ্ধতি হলো **Kadane's Algorithm**। এটি মাত্র একবার লুপ চালিয়েই ফলাফল বের করতে পারে।

লজিকটি কী?

এই অ্যালগরিদমটি প্রতিটি ইনডেক্সে এসে সিদ্ধান্ত নেয়: "আমি কি আগের যোগফলের সাথে থাকবো, নাকি এখান থেকে নতুন করে শুরু করবো?" যদি আগের যোগফল নেগেটিভ হয়ে যায়, তবে সেখান থেকে নতুন করে শুরু করাই ভালো।

Java Solution (Without Built-in Methods)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 3, -8, 7, -1, 2, 3};
        int n = arr.length;

        // ১. শুরুতে প্রথম এলিমেন্ট দিয়ে ভ্যারিয়েবল ডিক্লেয়ার করা
        int maxSoFar = arr[0];
        int currentMax = arr[0];

        // ২. দ্বিতীয় এলিমেন্ট (index 1) থেকে লুপ শুরু
        for (int i = 1; i < n; i++) {

            // ৩. currentMax আপডেট করার লজিক (Math.max এর বিকল্প)
            // যদি (আগের যোগফল + বর্তমান সংখ্যা) বর্তমান সংখ্যার চেয়ে বড় হয়
            if (currentMax + arr[i] > arr[i]) {
                currentMax = currentMax + arr[i];
            } else {
                // নাহলে বর্তমান সংখ্যা থেকেই নতুন সাব-অ্যারে শুরু হবে
                currentMax = arr[i];
            }

            // ৪. maxSoFar আপডেট করার লজিক
            if (currentMax > maxSoFar) {
                maxSoFar = currentMax;
            }
        }

        // আউটপুট প্রিন্ট করা
        System.out.println("Maximum Subarray Sum is: " + maxSoFar);
    }
}
```

==========

Problem 17

Maximum Product Subarray সমস্যাটি Maximum Sum (Kadane's) এর চেয়ে একটু আলাদা। কারণ, এখানে দুটি নেগেটিভ সংখ্যা গুণ হয়ে একটি বড় পজিটিভ সংখ্যা তৈরি করতে পারে। তাই আমাদের একই সাথে সবচাইতে বড় পজিটিভ গুণফল এবং সবচাইতে ছোট নেগেটিভ গুণফল—উভয়ই ট্র্যাক করতে হবে।

Java Solution (Efficient Approach)

আমরা কোনো বিল্ট-ইন মেথড ব্যবহার না করে ম্যানুয়ালি `if-else` দিয়ে সমাধানটি নিচে দেওয়া হলো:

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {-2, 6, -3, -10, 0, 2};
        int n = arr.length;

        // শুরুতে প্রথম এলিমেন্ট দিয়ে ভ্যারিয়েবল ডিক্লেয়ার করা
        long maxProduct = arr[0];
        long minProduct = arr[0];
        long result = arr[0];

        for (int i = 1; i < n; i++) {

            // যদি বর্তমান সংখ্যাটি নেগেটিভ হয়, তবে ম্যাক্স এবং মিন অদলবদল (Swap) হবে
            // কারণ নেগেটিভ দিয়ে গুণ করলে বড় সংখ্যা ছোট এবং ছোট সংখ্যা বড় হয়ে যায়
            if (arr[i] < 0) {
                long temp = maxProduct;
                maxProduct = minProduct;
                minProduct = temp;
            }

            // ম্যাক্স প্রোডাক্ট আপডেট করার লজিক
            if (arr[i] > maxProduct * arr[i]) {
                maxProduct = arr[i];
            } else {
                maxProduct = maxProduct * arr[i];
            }

            // মিন প্রোডাক্ট আপডেট করার লজিক (নেগেটিভ সংখ্যার জন্য)
            if (arr[i] < minProduct * arr[i]) {
                minProduct = arr[i];
```

```
            } else {
                minProduct = minProduct * arr[i];
            }

            // রেজাল্ট আপডেট
            if (maxProduct > result) {
                result = maxProduct;
            }
        }

        System.out.println("Maximum Product Subarray is: " + result);
    }
}
```

এই লজিকটি কেন কাজ করে?

1. নেগেটিভ সংখ্যার প্রভাব: যখনই আমরা একটি নেগেটিভ সংখ্যা পাই, আমাদের `maxProduct` এবং `minProduct` অদলবদল করতে হয়। কেন? কারণ ধরুন আপনার কাছে `max=10` আর `min=-20` আছে। এখন যদি `-2` দিয়ে গুণ করেন, তবে `max` হবে `-20` আর `min` হবে `40` (সবচাইতে বড়)। তাই সোয়াপ করা জরুরি।
2. জিরোর প্রভাব: যদি অ্যারেতে `0` থাকে, তবে উপরের লজিক অনুযায়ী গুণফল `0` হয়ে যাবে এবং পরের সংখ্যা থেকে আবার নতুন করে গুণ শুরু হবে।
3. তিনটি কেস হ্যান্ডেলিং:
    - পজিটিভ সংখ্যা আসলে গুণফল বাড়ে।
    - নেগেটিভ সংখ্যা আসলে সেটি `minProduct` কে পজিটিভ `maxProduct` এ পরিণত করতে পারে।
    - প্রতিটি ধাপে `result` আপডেট করে আমরা গ্লোবাল ম্যাক্সিমামটি ধরে রাখি।

==========

Problem 18

চমৎকার! এই সমস্যাটিকে বলা হয় **"Two Sum"** প্রবলেম। অর্থাৎ, একটি অ্যারে এবং একটি নির্দিষ্ট `targetSum` দেওয়া থাকবে; আপনাকে খুঁজে বের করতে হবে কোন দুটি সংখ্যার যোগফল সেই টার্গেটের সমান।

আমরা এটি **Nested Loop** (Brute Force) ব্যবহার করে সমাধান করবো, যেখানে কোনো বিল্ট-ইন মেথড লাগবে না।

Sample Test Case

- **Input Array:** `[10, 20, 35, 50, 75, 80]`
- **Target Sum:** `70`
- **Output:** `(20, 50)` কারণ $20 + 50 = 70$

Java Solution (Finding Pairs)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 15, 8, 5, 12, 20};
        int targetSum = 20;
        int n = arr.length;
        boolean found = false;

        System.out.println("Pairs with sum " + targetSum + " are:");

        // ১. প্রথম লুপটি একটি সংখ্যা নির্দিষ্ট করবে (i)
        for (int i = 0; i < n; i++) {

            // ২. দ্বিতীয় লুপটি তার পরের সংখ্যাগুলোর সাথে যোগ করে দেখবে (j = i + 1)
            for (int j = i + 1; j < n; j++) {

                // ৩. যদি দুটি সংখ্যার যোগফল টার্গেটের সমান হয়
                if (arr[i] + arr[j] == targetSum) {
                    System.out.println("(" + arr[i] + ", " + arr[j] + ")");
                    found = true;
                }
            }
        }

        // যদি কোনো পেয়ার না পাওয়া যায়
        if (!found) {
            System.out.println("No pairs found!");
        }
    }
}
```

কোডটি যেভাবে কাজ করছে:

1. **Nested Loop:** বাইরের লুপটি একটি সংখ্যা ধরে রাখে এবং ভেতরের লুপটি অ্যারের বাকি সংখ্যাগুলোর সাথে সেটি যোগ করে চেক করে।
2. **`j = i + 1` কেন?** আমরা `j` কে সব সময় `i` এর পরের ঘর থেকে শুরু করি যাতে একই পেয়ার দুইবার চেক না হয় (যেমন `15+5` একবার চেক হলে আবার `5+15` চেক করার দরকার নেই) এবং একই সংখ্যাকে নিজের সাথে যোগ না করা হয়।

3. **Time Complexity:** যেহেতু দুটি লুপ ব্যবহার করা হয়েছে, এর কর্মদক্ষতা $O(n^2)$। এটি ছোট অ্যারের জন্য বেশ কার্যকর।

==========

Problem 19

মেজরিটি এলিমেন্ট (Majority Element) বের করার সবচাইতে বিখ্যাত এবং দক্ষ অ্যালগরিদম হলো **Boyer-Moore Voting Algorithm**। এটি কোনো বাড়তি মেমরি ছাড়াই মাত্র একবার লুপ চালিয়ে সমাধান দিতে পারে।

মূল আইডিয়া:

যদি কোনো একটি এলিমেন্ট অর্ধেকের বেশি বার থাকে, তবে সেটিকে অন্য সব এলিমেন্ট মিলে "ভোট" দিয়েও বাদ দিতে পারবে না।

Java Solution (Boyer-Moore Voting Algorithm)

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 1, 2, 1, 3, 5, 1};
        int n = arr.length;

        // ধাপ ১: সম্ভাব্য মেজরিটি এলিমেন্ট খুঁজে বের করা (Candidate Selection)
        int candidate = -1;
        int count = 0;

        for (int i = 0; i < n; i++) {
            if (count == 0) {
                candidate = arr[i];
                count = 1;
            } else if (arr[i] == candidate) {
                count++;
            } else {
                count--;
            }
        }

        // ধাপ ২: নিশ্চিত হওয়া যে ক্যান্ডিডেটটি আসলেই মেজরিটি কি না
        int actualCount = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] == candidate) {
                actualCount++;
            }
```

```
        }

        if (actualCount > n / 2) {
            System.out.println("Majority Element: " + candidate);
        } else {
            System.out.println("No Majority Element Found (-1)");
        }
    }
}
```

কেন এই লজিকটি কাজ করে?

1. **Voting Phase:** আমরা একটি `candidate` ধরি। যখনই একই সংখ্যা পাই, আমরা `count` বাড়াই। অন্য সংখ্যা পেলে `count` কমাই। যদি `count` ০ হয়ে যায়, তার মানে আগের ক্যান্ডিডেট বাদ, বর্তমান সংখ্যাটি নতুন ক্যান্ডিডেট।
2. **Verification Phase:** ক্যান্ডিডেট পাওয়া গেলেই সে মেজরিটি হবে এমন কোনো কথা নেই (যেমন: `[1, 2, 3]` এর ক্ষেত্রে ক্যান্ডিডেট ৩ হতে পারে কিন্তু সে মেজরিটি নয়)। তাই দ্বিতীয়বার লুপ চালিয়ে গুনে দেখতে হয় সেটি আসলে $n/2$ এর চেয়ে বেশি বার আছে কি না।

==========

Problem 20

বাইনারি অ্যারেতে সবচাইতে লম্বা একই সংখ্যার **(0** অথবা **1)** সিকোয়েন্স খুঁজে বের করার জন্য আমাদের একটি লুপ চালিয়ে চেক করতে হবে যে বর্তমান সংখ্যাটি আগের সংখ্যার সমান কি না।

নিচে কোনো মেথড বা জটিল লাইব্রেরি ব্যবহার না করে একদম সাধারণ লজিক দিয়ে সমাধানটি দেওয়া হলো:

লজিক এবং ধাপসমূহ:

১. আমরা একটি `count` ভ্যারিয়েবল রাখবো যা বর্তমান সিকোয়েন্সের দৈর্ঘ্য গুনবে। ২. একটি `maxCount` ভ্যারিয়েবল রাখবো যা এখন পর্যন্ত পাওয়া সর্বোচ্চ দৈর্ঘ্যটি ধরে রাখবে। ৩. লুপের ভেতর যদি দেখি বর্তমান সংখ্যাটি তার আগের সংখ্যার সমান, তবে `count` বাড়িয়ে দেবো। ৪. আর যদি সমান না হয়, তবে সিকোয়েন্সটি ভেঙে গেছে; তাই `count` কে আবার ১ করে দেবো।

Java Solution

Java

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {0, 1, 0, 1, 1, 1, 1};
        int n = arr.length;
```

```
        // অ্যারে খালি থাকলে ০ প্রিন্ট করবে
        if (n == 0) {
            System.out.println(0);
            return;
        }

        int maxCount = 1; // অন্তত একটি সংখ্যা থাকলে দৈর্ঘ্য ১ হবেই
        int currentCount = 1;

        // ২য় সংখ্যা থেকে লুপ শুরু করা
        for (int i = 1; i < n; i++) {

            // যদি বর্তমান সংখ্যাটি আগেরটির সমান হয়
            if (arr[i] == arr[i - 1]) {
                currentCount++;
            } else {
                // সিকোয়েন্স আলাদা হলে কাউন্ট আবার ১ থেকে শুরু হবে
                currentCount = 1;
            }

            // সর্বোচ্চ দৈর্ঘ্যটি আপডেট করা (Math.max ছাড়া)
            if (currentCount > maxCount) {
                maxCount = currentCount;
            }
        }

        System.out.println("Longest contiguous sequence length: " + maxCount);
    }
}
================================================================
```